



Distributed Systems

TCP Server UNIX Signals

Ref: Unix Network Programming, The Socket Networking API by W Richard Stevens



Recap



TCP echo server and client

UDP echo server and client



1. Iterative or concurrent server?

- Most TCP servers are concurrent
- Most UDP servers are iterative

*recvfrom(...)**sendto(...)**~~recvfrom(...)~~ ?*

2. Each UDP socket has a receive buffer

// TCP ??

3. If the client datagram is lost

4. If the server reply is lost

non-blocking

- Client blocks in a call to `recvfrom`

TCP Echo Server

```

int main(int argc, char **argv)
{
    ...
    listenfd = socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);

    bind(listenfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
    listen(listenfd, 5);

    for ( ; ; ) {
        cllen = sizeof(cliaddr);
        connfd = accept(listenfd, (struct sockaddr*) &cliaddr, &cllen);

        if ( (childpid = fork()) == 0 ) { /* child process */
            close(listenfd); /* close listening socket */
            n = read(connfd, line, 512);
            write(connfd, line, n);
            close(connfd);
            exit(0);
        }
        //close(connfd); ?
    }
}

```

TCP Echo Client

```

int main(int argc, char **argv)
{
    int                sockfd;
    struct sockaddr_in  servaddr;

    if (argc != 2)
        err_quit("usage: tcpcli <IPaddress>");

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR);
    servaddr.sin_port = htons(SERV_PORT);

    connect(sockfd, (struct sockaddr*) &servaddr, sizeof(servaddr));

    fgets(sendline, 512, stdin);          /* do it all */
    n = strlen(sendline);
    write(sockfd, sendline, n);
    n = read(sockfd, recvline, 512);
    if (n < 0) ...
    recvline[n] = 0;
    fputs(recvline, stdout);
    exit(0);
}

```

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE



TCP Server

Points to remember



```
int getsockname (int sockfd, struct sockaddr *local,
socklen_t *addrlen);
```

- **after connect**, TCP client that does not call bind, can use getsockname to get the local IP address and local port number
- **after calling bind** with a port number of 0 TCP server can use getsockname to get local port number.
- A TCP server that binds wildcard IP address can use getsockname to obtain the local IP address assigned to the connection, **once a connection is established**

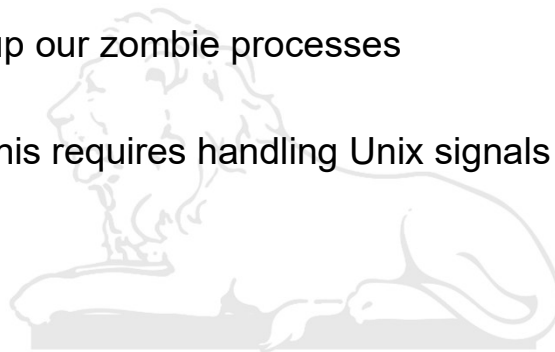
```
int getpeername (int sockfd, struct sockaddr *peeraddr,
socklen_t *addrlen);
```

- When a server is execed by the process that calls accept server can obtain the identity of the client by using getpeername

Zombie processes



- Clean up our zombie processes
- Doing this requires handling Unix signals



POSIX Signal Handling



- A *signal* is a notification to a process that an event has occurred.
- Signals are sometimes called **software interrupts**.
- Signals usually occur **asynchronously**.
- Signals can be sent
 - By one process to another process (or to itself)
 - By the kernel to a process
- The ^{SIGCHLD}SIGCHLD signal is **sent by the kernel** whenever a process terminates, to the parent of the terminating process.

IIT ROORKEE

9

POSIX Signal Handling



- Every signal has a *disposition*
- We set the *disposition* of a signal by calling the sigaction / signal function
- Three choices for the disposition:
 - Provide a function (signal handler) that is called whenever a specific signal occurs (**catching a signal**)


```
void handler(int signo);
```
 - We can *ignore* a signal by setting its disposition to SIG_IGN.
 - We can set the *default* disposition for a signal by setting its disposition to SIG_DFL.

IIT ROORKEE

10

POSIX Signal Handling



- The default is normally to terminate a process.
- Default disposition for SIGCHLD is to be ignored
- Two signals SIGKILL and SIGSTOP cannot be caught / ignored

Signal	Description	Default Action
SIGINT	terminal interrupt character	terminate
SIGCHLD	change in status of child	ignore
SIGKILL	termination	terminate
SIGSTOP	stop	Stop process

signal Function



- An easier way to set the disposition of a signal is to call the signal function
- Prototype for signal

→ `void (*signal (int signo, void (*func) (int)) (int));`

- Define the Sigfunc type

```
typedef void Sigfunc(int);
```

```
Sigfunc *signal (int signo, Sigfunc *func);
```

POSIX Signal Semantics

✓ standard
~~POSIX~~
 X POSIX real-time signals



- Once a signal handler is installed, it remains installed
 (On older systems?)
- While a signal handler is executing, the signal being delivered is blocked. (POSIX)
- Sigaction can be used to block additional signals specified in the mask.
- If a signal is generated one or more times while it is blocked, it is normally delivered only one time after the signal is unblocked (by default, Unix signals are not *queued*)
- It is possible to selectively block and unblock a set of signals using the sigprocmask function.

IIT ROORKEE

13

Handling Signal



```
void sighandler(int signum) {
    printf("Caught signal %d...\n", signum);
}

int main () {
    → signal(SIGINT, sighandler);
    while(1) {
        printf("Going to sleep for a second...\n");
        sleep(1);
    }
    return(0);
}
```

IIT ROORKEE

14

Handling Zombies



- The purpose of the zombie state
 - Information retained by the kernel includes the **process ID** of the child, its **termination status**, and information on the **resource utilization** of the child (CPU time, memory, etc.).
 - Whenever a process forks children it must wait for them
- Or
- Inform kernel that I am not interested in exit status (not for all variants of Unix)
 - Call wait in signal handler associated with SIGCHLD

Handling Zombies

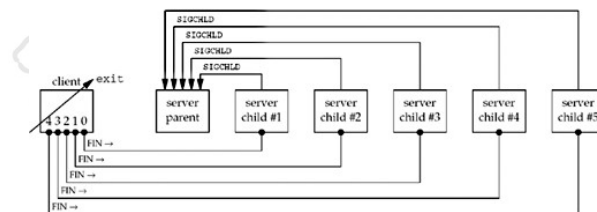


```
void sig_chld(int signo) {
    pid_t pid;
    int stat;
    pid = wait(&stat);
    printf("child %d terminated\\", pid);
    return;
}
```


Handling Zombies



- Client establishes five connections with the server and uses only one
- Client terminates
- All five connections are terminated at about the same time
- Server receives five FINs, one on each connection
- All five server children are terminated at about the same time
- Five **SIGCHLD** signals are delivered to the parent at about the same time



IIT ROORKEE 17

Handling Zombies



- Delivery of multiple occurrences of the same signal may cause the problem
- Establishing a signal handler and calling wait from that handler are insufficient for preventing zombies
- Solution: Call `waitpid` instead of `wait`

```
pid_t waitpid(pid_t pid, int *statloc, int options);
```

```
void sig_chld(int signo) {
    pid_t pid;
    int stat;
    while ( (pid = waitpid(-1, &stat, WNOHANG)) > 0)
        printf("child %d terminated\n", pid);
    return; }
```

IIT ROORKEE 18

Handling Zombies



1. We must catch the SIGCHLD signal when forking child processes.
2. We must handle interrupted system calls when we catch signals.
3. A SIGCHLD handler must be coded correctly using waitpid to prevent any zombies from being left around.



Design Issues

Issue 1: Time in Distributed Systems



Clock Synchronization

- Each process P_i has a state s_i which it transforms when it executes
- The state of the process includes the values of all its variables and objects in the system that it affects
- As a process executes it takes a series of actions (events)
- The sequence of events within a single process (and multiple processes on a single machine) can be totally ordered

Clock Synchronization

- In a centralized system the time is unambiguous
- Multiple machines with physical clocks in a Distributed System. How can we keep them more or less synchronized?